

Tài liệu Module AttentionClassifier

Lê Hoàng An

May 31, 2025

1 Giới thiệu

Tài liệu này mô tả các luồng xử lý của class `AttentionClassifier`, một lớp được thiết kế để phân loại trạng thái chú ý của khuôn mặt (Chú ý, Mất tập trung, Buồn ngủ) dựa trên các đặc trưng hình học. Class hỗ trợ ba loại mô hình: Random Forest, SVM, và Neural Network. Các phương thức chính bao gồm huấn luyện mô hình, dự đoán trạng thái, lưu/tải mô hình, và trực quan hóa kết quả (feature importance và confusion matrix). Tài liệu bao gồm các luồng chính, vòng đời (lifecycle), luồng xác thực đầu vào, và các luồng chi tiết liên quan đến quá trình phân loại.

2 Flow Chính của Module

Module `AttentionClassifier` hoạt động theo các bước chính sau:

1. **Khởi tạo:** Khởi tạo mô hình phân loại với loại mô hình (`random_forest`, `svm`, `neural_network`) và tùy chọn tải mô hình đã lưu.
2. **Huấn luyện mô hình:** Chuẩn hóa dữ liệu, huấn luyện mô hình trên tập dữ liệu, tối ưu hóa hyperparameters nếu yêu cầu, và đánh giá hiệu suất.
3. **Dự đoán:** Dự đoán trạng thái chú ý dựa trên các đặc trưng đầu vào, trả về nhãn, xác suất, và tên lớp.
4. **Lưu/tải mô hình:** Lưu mô hình đã huấn luyện hoặc tải mô hình từ file.
5. **Trực quan hóa:** Vẽ biểu đồ feature importance (nếu có) và confusion matrix để đánh giá mô hình.

3 Lifecycle của Module

Vòng đời của một đối tượng `AttentionClassifier` bao gồm các giai đoạn sau:

1. Khởi tạo (`__init__`):

- Nhận tham số `model_type` (`random_forest`, `svm`, `neural_network`) và `model_path` (đường dẫn đến mô hình đã lưu).
- Cài đặt thư viện `scikit-learn` nếu chưa có.
- Nếu `model_path` tồn tại, tải mô hình từ file; nếu không, khởi tạo mô hình mới bằng `_initialize_model`.

2. Khởi tạo mô hình (`_initialize_model`):

- Khởi tạo mô hình tương ứng với `model_type` với các tham số mặc định được tối ưu.
- Nếu `model_type` không được hỗ trợ, ném lỗi `ValueError`.

3. Huấn luyện (`train`):

- Chia dữ liệu thành tập huấn luyện và kiểm tra.
- Chuẩn hóa dữ liệu bằng `StandardScaler`.
- Tối ưu hóa hyperparameters nếu yêu cầu bằng `GridSearchCV`.
- Huấn luyện mô hình và đánh giá độ chính xác.
- Trực quan hóa feature importance (nếu có) và confusion matrix.

4. Dự đoán (`predict`):

- Nhận đặc trưng đầu vào, chuẩn hóa, và dự đoán trạng thái.
- Trả về nhãn, xác suất, và tên lớp.

5. Lưu/tải mô hình (`save_model`, `load_model`):

- Lưu mô hình, scaler, và các thông tin liên quan vào file.
- Tải lại mô hình và khôi phục trạng thái.

6. Kết thúc:

Không có phương thức hủy rõ ràng, đối tượng được giải phóng tự động bởi Python garbage collector khi không còn được tham chiếu.

4 Input Validation Flow

Luồng xác thực đầu vào được thực hiện như sau:

1. Khởi tạo (`__init__`):

- Kiểm tra thư viện `scikit-learn`: Nếu chưa cài đặt, tự động cài đặt bằng `pip`.
- Kiểm tra `model_path`: Nếu được cung cấp, kiểm tra sự tồn tại của file bằng `os.path.exists`.
- Kiểm tra `model_type`: Được kiểm tra trong `_initialize_model`, ném `ValueError` nếu không phải `random_forest`, `svm`, hoặc `neural_network`.

2. Huấn luyện (`train`):

- Kiểm tra kích thước dữ liệu: Nếu số mẫu của `X` không khớp với `y`, ném `ValueError`.
- Kiểm tra `validation_split`: Giả định giá trị nằm trong `[0, 1]`, không có kiểm tra rõ ràng (có thể gây lỗi nếu không hợp lệ).
- Kiểm tra `optimize_hyperparams`: Nếu `True`, thực hiện `GridSearchCV`.

3. Dự đoán (`predict`):

- Kiểm tra mô hình: Nếu `self.model` là `None`, trả về giá trị mặc định (nhân 0, xác suất 0.0, lớp Chú ý).
- Kiểm tra định dạng features: Chuyển đổi list thành mảng NumPy và reshape nếu cần (1D thành 2D).
- Kiểm tra scaler: Nếu đã huấn luyện, áp dụng chuẩn hóa; nếu không, bỏ qua.
- Xử lý lỗi: Bắt ngoại lệ trong quá trình dự đoán và trả về giá trị mặc định.

4. Lưu mô hình (`save_model`):

- Kiểm tra `self.model`: Nếu là `None`, in thông báo lỗi và thoát.
- Kiểm tra `model_path`: Tạo thư mục chứa nếu chưa tồn tại bằng `os.makedirs`.

5. Tải mô hình (`load_model`):

- Giả định file `model_path` hợp lệ (đã được kiểm tra trước đó trong `__init__`).
- Xử lý lỗi tiềm ẩn: Nếu file không chứa dữ liệu hợp lệ, có thể gây lỗi khi tải bằng `pickle.load`.

5 Các Flow Chi Tiết

Dưới đây là các luồng chi tiết liên quan đến các phương thức chính:

5.1 Khởi tạo mô hình (`_initialize_model`)

- **Đầu vào:** Không có (sử dụng `self.model_type`).
- **Xử lý:**
 - Nếu `model_type='random_forest'`, khởi tạo `RandomForestClassifier` với 200 cây, độ sâu tối đa 15, và các tham số tối ưu khác.
 - Nếu `model_type='svm'`, khởi tạo `SVC` với kernel RBF, `C=10.0`, và xác suất đầu ra.
 - Nếu `model_type='neural_network'`, khởi tạo `MLPClassifier` với 3 lớp ẩn (128, 64, 32), kích hoạt ReLU, và early stopping.
 - Nếu `model_type` không hợp lệ, ném `ValueError`.
- **Đầu ra:** Mô hình được gán vào `self.model`.

5.2 Huấn luyện mô hình (`train`)

- **Đầu vào:** Dữ liệu đặc trưng `X`, nhãn `y`, cờ tối ưu hóa `optimize_hyperparams`, tỷ lệ chia kiểm tra `validation_split`.
- **Xử lý:**
 - Kiểm tra số mẫu `X` và `y` khớp nhau.
 - Chia dữ liệu thành tập huấn luyện và kiểm tra bằng `train_test_split` với `stratify=y`.
 - Chuẩn hóa dữ liệu bằng `StandardScaler`.
 - Nếu `optimize_hyperparams=True`, gọi `_optimize_hyperparameters` để tìm tham số tốt nhất.

- Huấn luyện mô hình trên `X_train_scaled`.
- Đánh giá độ chính xác trên tập huấn luyện và kiểm tra.
- In báo cáo phân loại bằng `classification_report`.
- Nếu mô hình hỗ trợ (Random Forest), lưu và vẽ feature importance.
- Vẽ confusion matrix bằng `_plot_confusion_matrix`.
- **Đầu ra:** Độ chính xác trên tập kiểm tra.

5.3 Tối ưu hóa hyperparameters (`_optimize_hyperparameters`)

- **Đầu vào:** Dữ liệu huấn luyện `X_train`, nhãn `y_train`.
- **Xử lý:**
 - Định nghĩa lưới tham số cho từng loại mô hình (`random_forest`, `svm`, `neural_network`).
 - Sử dụng `GridSearchCV` với 5-fold cross-validation để tìm tham số tốt nhất.
 - In tham số tốt nhất và điểm số cross-validation tốt nhất.
- **Đầu ra:** Estimator tốt nhất từ `GridSearchCV`.

5.4 Vẽ feature importance (`_plot_feature_importance`)

- **Đầu vào:** Không có (sử dụng `self.feature_importance`).
- **Xử lý:**
 - Nếu `self.feature_importance` là `None`, thoát.
 - Vẽ biểu đồ thanh với mức độ quan trọng của các đặc trưng (EAR, MAR, Head_Pose_X, Head_Pose_Y, Face_Ratio, Eye_Distance).
 - Sắp xếp theo mức độ quan trọng giảm dần.
- **Đầu ra:** Biểu đồ thanh hiển thị trên màn hình.

5.5 Vẽ confusion matrix (`_plot_confusion_matrix`)

- **Đầu vào:** Nhãn thực tế `y_true`, nhãn dự đoán `y_pred`.

- **Xử lý:**
 - Tính confusion matrix bằng `confusion_matrix`.
 - Vẽ heatmap bằng `seaborn.heatmap` với nhãn lớp (Chú ý, Mất tập trung, Buồn ngủ).
- **Đầu ra:** Biểu đồ heatmap hiển thị trên màn hình.

5.6 Dự đoán (**predict**)

- **Đầu vào:** Đặc trưng features (list hoặc mảng NumPy).
- **Xử lý:**
 - Nếu mô hình chưa được huấn luyện, trả về giá trị mặc định (nhãn 0, xác suất 0.0, lớp Chú ý).
 - Chuyển đổi features thành mảng NumPy 2D.
 - Chuẩn hóa features nếu `self.scaler` tồn tại.
 - Dự đoán nhãn và xác suất bằng `model.predict` và `model.predict_proba`.
 - Xử lý lỗi: Nếu có ngoại lệ, trả về giá trị mặc định.
- **Đầu ra:** Tuple (nhãn, xác suất, tên lớp).

5.7 Lưu và tải mô hình (**save_model, load_model**)

- **Đầu vào:** `model_path` (đường dẫn file).
- **Xử lý:**
 - `save_model`: Lưu model, scaler, model_type, classes, và feature_importance vào file bằng pickle.
 - `load_model`: Tải dữ liệu từ file và khôi phục trạng thái các thuộc tính.
- **Đầu ra:** Không có (chỉ in thông báo).

6 Kết luận

Class `AttentionClassifier` cung cấp một giải pháp linh hoạt để phân loại trạng thái chú ý với ba mô hình học máy (Random Forest, SVM, Neural

Network). Các luồng xử lý được thiết kế để đảm bảo tính mạnh mẽ, bao gồm xác thực đầu vào, chuẩn hóa dữ liệu, tối ưu hóa hyperparameters, và trực quan hóa kết quả. Để sử dụng hiệu quả, cần đảm bảo các thư viện (scikit-learn, matplotlib, seaborn, numpy) được cài đặt và dữ liệu đầu vào được chuẩn bị đúng cách.